LoopNav: Benchmarking Spatial Consistency in World Models

Kewei Lian NUS
Shaofei Cai Peking Univeristy
Yitao Liang Peking Univeristy
Anji Liu NUS

Abstract

The ability to simulate the world in a spatially consistent manner is a crucial requirement for effective world models. Such a model enables high-quality visual generation, and also ensures the reliability of world models for downstream tasks such as simulation and planning. It must not only retain long-horizon observational information, but also enables the construction of explicit or implicit internal spatial representations. However, existing datasets do not explicitly enforce spatial consistency constraints, limiting both the ability to systematically evaluate this capability and to learn it through data-driven approaches. Furthermore, most existing benchmarks primarily emphasize visual coherence or generation quality, neglecting the requirement of long-range spatial consistency. To bridge this gap, we propose LOOPNAV, a dataset and corresponding benchmark centered on loop-based navigation for evaluating spatial consistency. The dataset comprises 250 hours of loop-based navigation videos with actions, collected from diverse locations in the open-world environment of Minecraft. We further introduce a Scene Graph Consistency Score to quantify spatial consistency while remaining invariant to pixel-level variations. Dataset, benchmark, and code are open-sourced to support future research.

1 Introduction

Although recent advances in world models have significantly improved the visual quality of generated observations [Zhu et al., 2024, Ding et al., 2024, Brooks et al., 2024, Bruce et al., 2024], these models often struggle to preserve spatial consistency over extended rollouts [Decart et al., 2024, Guo et al., 2025]. Spatial consistency, the ability to preserve coherent and stable spatial structures across time, is essential for the reliability of world models in downstream applications such as model-based reinforcement learning [Bar et al., 2024, Alonso et al., 2024, Valevski et al., 2024], autonomous driving [Wang et al., 2023a, Gao et al., 2024], and model predictive control [Du et al., 2023, Hafner et al., 2020, 2024, Yang et al., 2023, 2024]. When violated, it can lead to severe failure modes: hallucinated structures, contradictions with prior observations, and visually incoherent scenes. For instance, in a world model guided navigation task, inconsistent rendering of revisited locations impedes global planning and loop closure.

A key challenge underlying spatial inconsistency is the need to retain and utilize information over long temporal horizons. In principle, addressing this challenge requires models to maintain representations of previously observed spatial structures and re-anchor them upon revisitation. However, mainstream Transformer-based architectures are typically limited to short context windows due to the quadratic scaling of the attention mechanism [Vaswani et al., 2017], making it difficult to directly incorporate long-range dependencies spanning hundreds or thousands of frames.

To mitigate this limitation, prior work has explored augmenting world models with memory mechanisms that enable long-term information retention [Graves et al., 2014]. These mechanisms may take various forms, including explicit spatial representations such as maps [Gupta et al., 2019, Yang et al.,

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

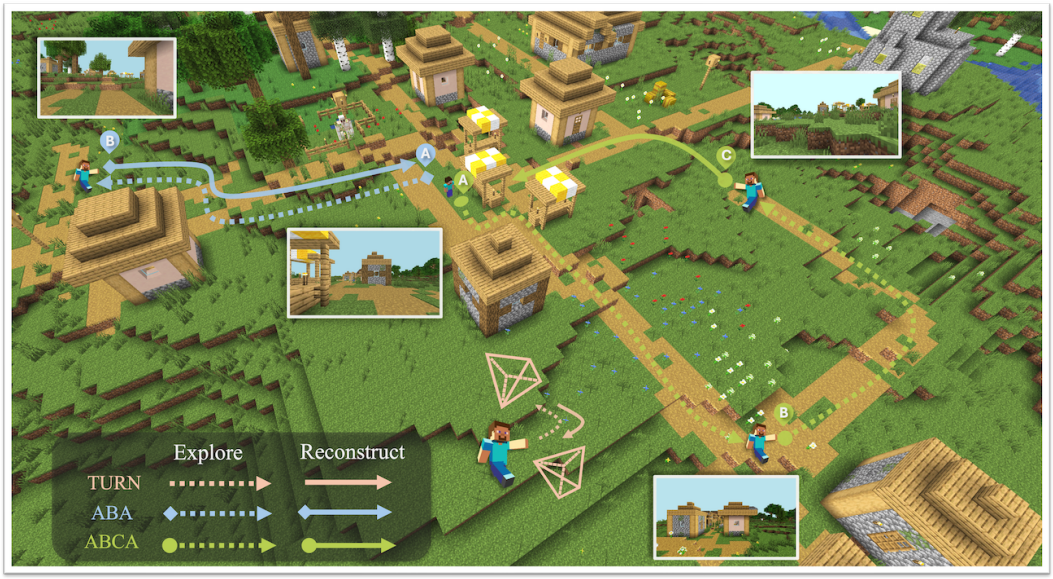


Figure 1: **The Loop-Style Trajectory Data for Training and Benchmarking**. To be able to explicitly enforce spatial consistency, our dataset follows a loop-style navigation trajectory, including TURN, ABA and ABCA types. Our benchmark design explicitly separates the generation phase from the exploration phase and only evaluates the video quality of the generation part.

2025], coordinate systems [Wang et al., 2023b], or graph-based structures [Savinov et al., 2018, Kim et al., 2022], as well as implicit representations based on learned embeddings or attention [Parisotto and Salakhutdinov, 2017, Wang* et al., 2025]. While these approaches provide promising directions, they highlight a broader challenge: enabling models to maintain coherent spatial representations over long horizons remains an open problem.

Despite its importance, spatial consistency remains underexplored, particularly in terms of how it is supported, learned, and evaluated. A major reason for this gap is the absence of datasets that explicitly demand spatial consistency. Most existing datasets are designed for open-ended exploration, where agents continually encounter novel structures and objects while rarely revisiting previously seen locations. As a result, world models trained on such data tend to rely on global environmental priors to generate plausible observations, rather than leveraging episodic memory to reconstruct previously observed content. Meanwhile, evaluation metrics often prioritize visual fidelity and short-term temporal smoothness over long-term spatial coherence or logical consistency. This leads to models that function more as visually impressive but impractical "dream machines", limiting their applicability in downstream decision-making tasks.

Therefore, we argue that such a dataset must feature looped trajectories that revisit the same locations and objects from diverse viewpoints. We construct such a dataset, LOOPNAV, in the open-world environment of Minecraft. The dataset comprises over 250 hours of **loop-style** navigation trajectories across 147 diverse locations. Each trajectory follows a loop exploration pattern, ensuring that the same locations and layouts are revisited from varying camera views and times. This loop-based data structure naturally incentivizes models to learn long-horizon spatial consistency. Besides, we carefully select positions with diverse landmark objects to ensure that observations from different locations are **visually distinguishable**. To support progressive learning, the dataset also includes a **curriculum of sequence lengths**, enabling models to transition from short-term to long-term tasks.

For benchmarking, we adopt an **explore-then-generate** protocol to evaluate spatial consistency. Specifically, we sample a loop trajectory (e.g., $A \rightarrow B \rightarrow A$), where the first segment ($A \rightarrow B$) is provided as context, and the remaining segment ($B \rightarrow A$) serves as the generation target. Existing evaluation metrics, such as SSIM and LPIPS [Zhang et al., 2018], primarily assess visual quality. However, in world model generation, even when conditioned on identical action sequences, the generated trajectory may exhibit slight deviations—such as small translations, scaling, or accumulated drift—due to the inherent stochasticity of action execution and cumulative errors over time. These metrics can be sensitive to such misalignments, failing to reflect true spatial consistency.

To address this limitation, we introduce the **Scene Graph Consistency Score** (SGCS), an object-centric metric for evaluating spatial consistency. Specifically, video frames are first decomposed into category-wise object masks, and consistency is measured through object-level matching based on semantic category and spatial alignment. By comparing the matched object structures across trajectories, SGCS abstracts away pixel-level differences while remaining sensitive to semantic inconsistencies such as object disappearance, replacement, and large spatial layout changes.

In summary, our contributions are as follows:

- **A dataset and benchmark for spatial consistency.** We introduce LOOPNAV, a dataset and benchmark centered on loop-based navigation, which bridges the gap between existing world model training setups and the need to evaluate long-horizon spatial consistency. It provides a unified platform for both training and evaluation under explicit spatial consistency constraints.
- **A structure-aware metric for spatial consistency.** We propose the Scene Graph Consistency Score (SGCS), a structure-aware metric that evaluates spatial consistency while abstracting away pixel-level variations. It is robust to minor geometric misalignments, while remaining sensitive to semantic inconsistencies.
- **An open-source data collection framework.** We release a scalable and flexible pipeline for collecting navigation trajectories in Minecraft, which can be readily extended to other data collection settings. We also open-source the dataset and evaluation code to facilitate future research on spatial consistency in world models. Dataset, benchmark, and code are available [1] [2].

2 Related Work

2.1 World Models

The goal of a world model is to simulate the environment: Given the current state and action, it predicts the next state and the corresponding reward. World models were originally proposed to improve sample efficiency in reinforcement learning [Oh et al., 2015, Ha and Schmidhuber, 2018, Hafner et al., 2020, 2022, 2024, Robine et al., 2023, Alonso et al., 2024]. Different architectures, to varying degrees, retain information from the past to aid in future image prediction; however, they still lack an effective memory design capable of maintaining long-horizon spatial consistency.

Beyond model-based reinforcement learning, the potential of world models has also been increasingly explored in other domains such as game engines [Menapace et al., 2021, Valevski et al., 2024, Decart et al., 2024], autonomous driving [Hu et al., 2023, Wang et al., 2023a, Gao et al., 2024], robot manipulation [Yang et al., 2023, Wu et al., 2022, Bar et al., 2024]. In those downstream tasks, spatial consistency is even more important.

2.2 Minecraft as an AI Testbed

Minecraft is an open-world environment characterized by diverse terrains and rich interaction dynamics. Recent research has increasingly adopted Minecraft as a platform for training generative agents [Baker et al., 2022, Lifshitz et al., 2024, Cai et al., 2023, 2024, 2025b,a] and constructing digital world models [Guo et al., 2025, Decart et al., 2024, Hong et al., 2024, Song et al., 2025].

A more comprehensive review of related work is provided in the Appendix E.

3 LOOPNAV Dataset

3.1 Environment

We collect data in the open-world environment of Minecraft, which provides a suitable platform for large-scale and controlled data generation. Its procedurally generated worlds offer diverse and

[1] Dataset page: https://huggingface.co/datasets/kevinLian/LoopNav
[2] Code page: https://github.com/Kevin-lkw/LoopNav

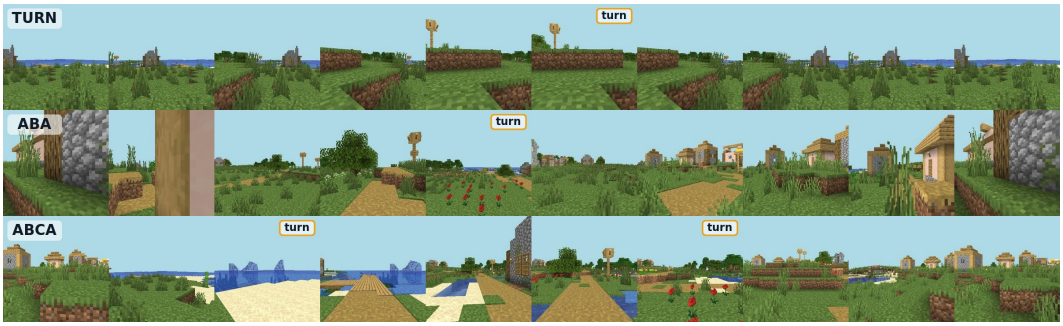


Figure 2: **Demostration of our dataset.** Rows show TURN, ABA , and ABCA , with time progressing from left to right. Ten frames are sampled from each trajectory, with turning points marked by orange turn labels.

non-repetitive environments, while its mature ecosystem enables flexible customization and extension. Compared to real-world data collection, it also allows efficient, scalable, and highly parallelizable simulation. Further details are provided in Appendix A.1.

3.2 Collection Pipeline

We propose three principles for trajectory data collection: *visual discriminability*, *loop closure*, and *curriculum-based progression*. Below, we detail each principle, followed by our implementation.

Principle 1: Visual Discriminability

The sequence of observations along the trajectory should be visually distinguishable over time to capture meaningful scene variation.

Within this procedurally generated world, we identify 6 categories comprising a total of 120 distinct villages, covering diverse terrains and architectural layouts. As players navigate through a village, visually distinctive structures such as houses, farms, and pathways serve as salient landmarks for spatial reasoning and cognitive map construction. The "visually distinguishable" principle is particularly important for evaluating spatial consistency. If a scene lacks distinctive objects or structural variation, visually different states may become difficult to distinguish, making it ambiguous whether a generated frame is truly spatially consistent or merely visually similar. By ensuring sufficient variation in object appearance and spatial layout, our dataset enables more reliable evaluation of long-horizon spatial consistency. Detailed descriptions are provided in Appendix B.

Principle 2: Loop Closure

The generated trajectories must form at least one spatial loop, ensuring repeated visits to the same locations.

We propose three types of trajectory structures: TURN, $A \to B \to A$ and $A \to B \to C \to A$, where A, B, and C denote distinct locations within the environment.

We use the ABA trajectory as a representative case to illustrate our data collection process. As detailed in Algorithm 1, we first select a location S from one of our collected sites,then randomly sample a starting point A within the start range of S. This is done to ensure diversity in the starting positions within the same location. A target location B is sampled within a specified navigation range. Trajectory recording begins: the agent performs a full 360-degree rotation at location A to observe the surrounding environment and establish spatial context. It then navigates from A to B using the A* algorithm, and subsequently returns from B to A, completing the loop. To reduce trajectory variance caused by independently planning the $A \to B$ and $B \to A$ paths with A*, we make sure the return path follows the recorded forward waypoints in reverse order. Details are provided in Appendix B.3. We put visualization of our dataset, including frames and bird-eye view in Appendix B.4.

Algorithm 1 Loop Navigation Data Collection

Require: Location S; ranges R_{st}, R_{nav}
Ensure: Loop trajectory T
$A \leftarrow \textsc{SamplePoint}(S, R_{\text{st}})$
$B \leftarrow \textsc{SamplePoint}(A, R_{\text{nav}})$
Teleport the agent to A and rotate by $360°$
$T_{AB} \leftarrow \textsc{Navigate}(A, B)$
$T_{BA} \leftarrow \textsc{Navigate}(B, A)$
return $\textsc{Concat}(T_{AB}, T_{BA})$

function $\textsc{SamplePoint}(C, r)$
  repeat
    $\Delta \sim \mathcal{U}([-r, r]^2)$
    $P \leftarrow C + \Delta$
  until $\|\Delta\|_2 \geq 0.8r \wedge \textsc{IsValid}(P)$
  return P
end function

Principle 3: Curriculum-based Progression

The trajectories follow a curriculum design, gradually expanding the spatial exploration radius to support progressive learning and evaluation.

According to the exploration radius, we define four difficulty levels: 5, 15, 30, and 50 blocks. To improve training robustness and introduce variability, the target location B is not sampled at a fixed distance. Instead, for a difficulty level x, we sample distances within the interval $[0.8x, \sqrt{2}x]$, ensuring diverse trajectory lengths while minimizing overlap between different difficulty levels. The full sampling procedure is also detailed in Algorithm 1. Collect dataset trajectory length is visualized in Figure 7.

3.3 Implementation Details

We collect 20 trajectories for each combination of location and exploration radius. The final dataset contains approximately 19.2 million frames, amounting to roughly 250 hours of navigation trajectories.

We collect navigation trajectories in Minecraft using a customized data collection pipeline built on Mineflayer. Further details on the data collection system are provided in Appendix A. Action design, and collected data format are in Appendix B.

4 Benchmark

4.1 Loop-Based Evaluation Protocol

Our benchmark is designed around $A \to B \to A$ loop trajectories. In this setting, we treat the $A \to B$ segment as an exploration phase, which serves as the contextual input to the model, and the $B \to A$ segment as a reconstruction phase, where the agent returns to a previously visited location. We provide an illustration of these phases in Figure 1. As such, we evaluate model performance only on the $B \to A$ segment, as it provides a clear test of the model's ability to maintain spatial consistency during long-horizon rollouts. Similarly, for longer $A \to B \to C \to A$ trajectories, we evaluate only the $C \to A$ segment. A detailed discussion is on Appendix C.3.

4.2 Limitations of Visual Quality Metrics

Given aligned ground-truth and generated trajectories, a natural approach is to evaluate their similarity using existing video generation metrics. However, such metrics are not well suited for assessing spatial consistency in world models. The generated trajectory may exhibit small geometric deviations due to accumulated errors in action execution. These deviations can manifest as slight translations, scaling, or viewpoint shifts over long rollouts. Importantly, such perturbations do not necessarily indicate a failure of spatial consistency. However, visual quality metrics are highly sensitive to these geometric perturbations. They may penalize predictions that are semantically consistent but not perfectly aligned with the ground truth, leading to low scores. At the same time, these metrics fail to capture spatial consistency itself. They primarily assess visual fidelity or perceptual similarity, rather than whether the generated trajectory remains consistent with previously observed scenes. Consequently, a visually plausible but spatially inconsistent generation may still receive a high score.

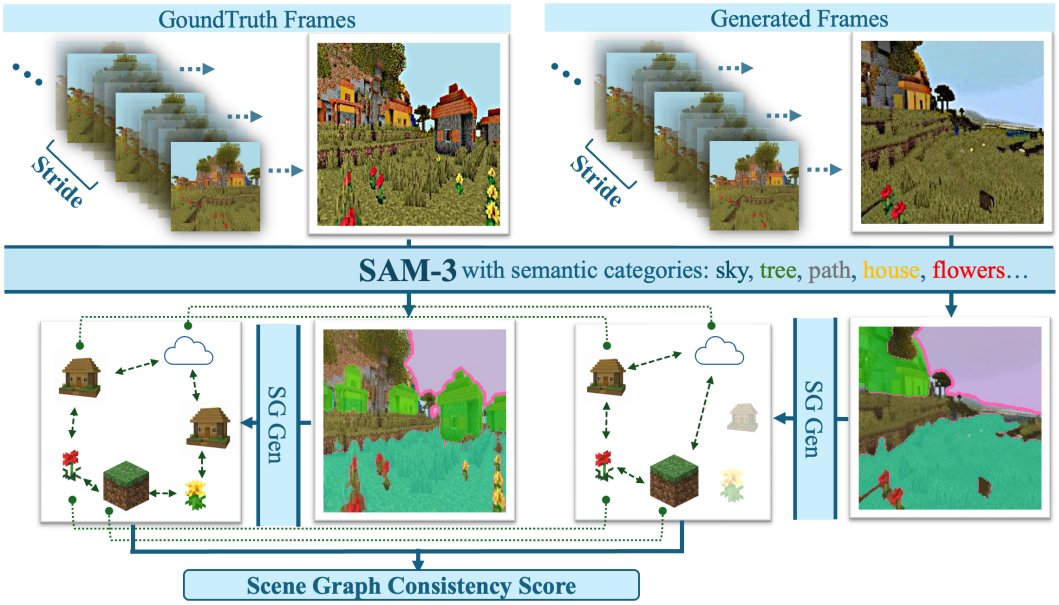


Figure 3: **Evaluation pipeline: Scene Graph Consistency Score (SGCS).** We sample aligned frames from generated and ground-truth videos, extract category-wise object masks using SAM 3, and perform bipartite matching based on centroid distance. The final score aggregates category-level consistency with area weighting. SGCS is robust to geometric misalignment while capturing inconsistencies in object presence and spatial layout.

4.3 Scene Graph Consistency Score

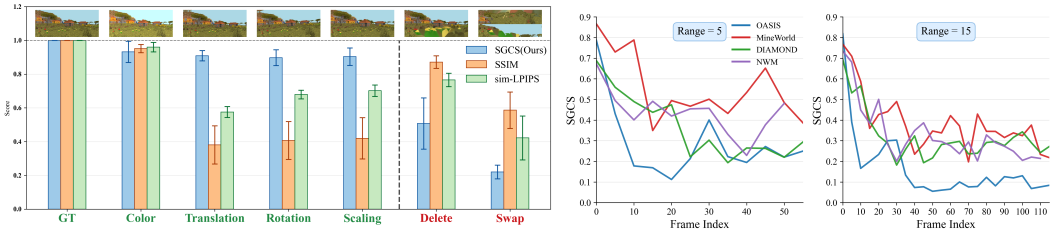
To address the limitations of pixel-level metrics, we introduce the **Scene Graph Consistency Score (SGCS)**. SGCS is designed to be robust to minor geometric perturbations, such as translation, rotation, and scaling, while remaining sensitive to changes in spatial consistency, including object disappearance, replacement, and incorrect spatial relationships.

Figure 3 illustrates our evaluation pipeline. Given a generated video and its corresponding ground-truth video of equal length, we first sample aligned frame pairs at a fixed temporal stride. For each pair of frames, we compute SGCS based on category-wise object matching.

Specifically, for each semantic category c (e.g., building, tree, flower, grass), we segment both frames into object masks using SAM 3 [Carion et al., 2026]. We compute the centroid of each mask and identify matchable pairs based on their normalized spatial distance. Intuitively, two masks are considered a valid match if their centers are sufficiently close relative to the image size.

Since multiple instances of the same object category may appear in a frame, we perform bipartite matching (via the Hungarian algorithm [Kuhn, 1955]) to obtain the maximum number of matched object pairs. Based on the number of matched and unmatched masks, we compute a category-level consistency score that reflects how well objects of category c are preserved. To aggregate across categories, we weight each category by its average visible area in the two frames, giving more importance to dominant structures. The final frame-level SGCS is obtained as the weighted average over all categories.

For video-level evaluation, we compute SGCS for all sampled frame pairs and report their average. Importantly, SGCS abstracts away pixel-level differences and instead evaluates consistency at the level of object presence and spatial arrangement. As a result, it is robust to minor geometric misalignments, while remaining sensitive to spatial inconsistencies. We leave the details of implimentation in Appendix C.1.



(a) Controlled perturbation comparison.

(b) Per-frame SGCS over time.

Figure 4: **Left: Comparison of metrics under controlled perturbations.** From left to right, Geometric Perturbations: ground truth, color change, small translation, small rotation, and scaling; Spatial Inconsistencies: object deletion and object position swapping. We compare SGCS (ours), SSIM, and sim-LPIPS. Higher values indicate better. **Right: Per-frame SGCS.** Spatial consistency degrades across all baselines as generation progresses.

5 Experiments

5.1 SGCS Metric Validation

We evaluate SGCS under controlled perturbations to validate its robustness and sensitivity. Specifically, we apply geometric perturbations (colour change, translation, rotation, scaling) and semantic modifications (object deletion and swapping) to ground-truth videos. We compare SGCS, SSIM, and sim-LPIPS (defined as 1-LPIPS). All metrics are normalized such that higher values indicate better consistency and GT achieves a score of 1. As shown in Figure 4a, SGCS remains stable under geometric perturbations, demonstrating robustness to pixel-level misalignment. In contrast, SSIM and LPIPS degrade significantly under some transformations.

Conversely, under semantic modifications, SGCS decreases sharply, correctly identifying violations of spatial consistency, while SSIM and LPIPS may still assign relatively high scores due to preserved visual appearance.

These results confirm that SGCS achieves the desired balance between invariance to geometric perturbations and sensitivity to spatial inconsistencies. We leave more demonstrations of synthetic perturbation on Figure 10 and the details of implementation on Appendix D.1.

5.2 World Model Baselines

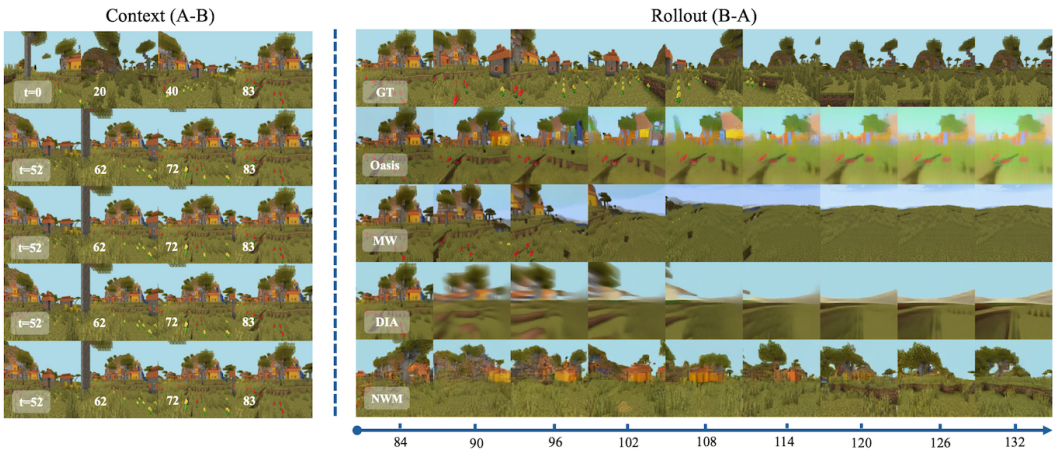


Figure 5: **Qualitative Result of Four Baselines**. Top to buttom: Ground Truth(GT), Oasis, Mineworld(MW), DIAMOND(DIA), Navigation World Model(NWM). Leftmost label "t=52" indicates the start context range accepted by each model. All models begin rollout from frame 84.

Table 1: SGCS (↑) across TURN, ABA, and ABCA. Results are reported as mean ± standard deviation over 60 trajectories. Bold means indicate the best results.

Task	Range	Oasis	NWM	MineWorld	DIAMOND	WorldMem
TURN	60°–90°	0.555 ± 0.21	0.344 ± 0.23	0.721 ± 0.17	0.597 ± 0.23	$\mathbf{0.839} \pm 0.14$
	90°–120°	0.458 ± 0.22	0.293 ± 0.19	0.665 ± 0.21	0.484 ± 0.23	$\mathbf{0.846} \pm 0.12$
	120°–150°	0.472 ± 0.24	0.291 ± 0.20	0.597 ± 0.20	0.470 ± 0.26	$\mathbf{0.839} \pm 0.13$
	150°–180°	0.458 ± 0.20	0.253 ± 0.16	0.593 ± 0.18	0.510 ± 0.23	$\mathbf{0.829} \pm 0.11$
ABA	5	0.308 ± 0.24	0.398 ± 0.26	0.413 ± 0.24	0.311 ± 0.26	$\mathbf{0.530} \pm 0.25$
	15	0.263 ± 0.20	$\mathbf{0.382} \pm 0.24$	0.367 ± 0.23	0.306 ± 0.23	0.373 ± 0.21
	30	0.184 ± 0.15	$\mathbf{0.376} \pm 0.21$	0.295 ± 0.24	0.198 ± 0.19	0.195 ± 0.15
	50	0.110 ± 0.09	$\mathbf{0.352} \pm 0.21$	0.325 ± 0.23	0.181 ± 0.18	0.108 ± 0.07
ABCA	5	0.352 ± 0.28	0.390 ± 0.30	0.451 ± 0.26	0.349 ± 0.26	$\mathbf{0.551} \pm 0.27$
	15	0.273 ± 0.22	0.402 ± 0.25	$\mathbf{0.421} \pm 0.24$	0.276 ± 0.22	0.415 ± 0.25
	30	0.205 ± 0.21	$\mathbf{0.365} \pm 0.25$	0.330 ± 0.23	0.237 ± 0.23	0.291 ± 0.25
	50	0.204 ± 0.21	$\mathbf{0.391} \pm 0.22$	0.376 ± 0.26	0.218 ± 0.23	0.329 ± 0.27

We evaluate the following baselines on our dataset and benchmark. We set the context length of all baselines to 32 frames.

Oasis [Decart et al., 2024] is a world model with a ViT [Dosovitskiy et al., 2021] as spatial autoencoder, and a DiT [Peebles and Xie, 2022] as latent diffusion backbone. We use the pretrained checkpoint. **Mineworld** [Guo et al., 2025] is another interactive world model driven by a visual-action autoregressive Transformer pretrained on Minecraft. We use the pretrained checkpoint. **DIAMOND** [Alonso et al., 2024] is a diffusion-based world model built upon the UNet architecture [Ronneberger et al., 2015]. We pretrain a model using our dataset from scratch. **Navigation World Model** [Bar et al., 2024] is a controllable video generation model that predicts future visual observations conditioned on past observations and navigation actions. We pretrain NWM model using our dataset from scratch. The implementation details and additional results of the four baselines are provided in the Appendix D.

5.3 Experiment Settings

Although our dataset includes data from villages, biomes, and structures, we focus solely on village environments for training and evaluation. This decision is based on the large number of villages in Minecraft and the inherent diversity of them. We use traject number 1-16 as training set, 17-18 as validation set and 19-20 as test set.

Evaluation We evaluate each model across all navigation ranges. Due to the large scale of the test dataset, we do not perform evaluation on the entire set. Instead, we sample the first three trajectories in lexicographic order from each of the 6 villages with index 20, resulting in 18 evaluation trajectories per navigation range. Each trajectory is assessed using the metrics described in the benchmark section. We report the average performance across all 18 trajectories, as shown in Table 2. In addition, we randomly select one trajectory to present qualitative results, shown in Figure 5.

5.4 Analysis

Overall Performance Overall, all four baselines exhibit unsatisfactory performance in maintaining spatial consistency. As shown in Figure 4b and Figure 5, MineWorld and NWM are able to generate approximately spatially consistent frames during the first ∼20 frames, but fail to preserve spatial consistency in later stages. In contrast, Oasis and Diamond quickly degrade into severe visual artifacts, eventually collapsing into blurry and incoherent outputs, especially as the generation horizon increases. In these cases, the models gradually deteriorate from minor imperfections to complete visual failure over time. As shown in Figure 4b, the severe visual distortions in Oasis lead to consistently low `SGCS` scores. This observation indicating that `SGCS` aligns well with visual perception.

Metric Fidelity to Model Performance and Task Difficulty As shown in Table 2, our metric exhibits strong correlation with perceived visual quality. MineWorld achieves the best performance, followed by NWM and DIAMOND, while Oasis performs the worst. This ranking is consistent with human qualitative observations. Furthermore, the results reveal a clear difficulty gradient in our task design: as the navigation range increases, the task becomes more challenging, leading to a consistent decrease in SGCS.

In contrast, traditional visual quality metrics, including SSIM, LPIPS, and FVD, fail to faithfully reflect the relative performance of the baseline models and do not capture the increasing task difficulty. These observations are consistent with our earlier findings, highlighting the limitations of conventional visual similarity metrics in evaluating spatial consistency.

Outlier Analysis We observe a single outlier where MineWorld and DIAMOND achieve higher performance at ABCA type navgation, trajectory length 50 compared to length 30. This occurs because, at length 50, the task corresponds to extremely long-horizon navigation (approximately 1200 frames; see Fig. 7). Additionaly, ABCA trajectories involve more extensive exploration and wandering, often leading the agent to move beyond the village. In such scenarios, many frames encountered during navigation consist primarily of simple backgrounds, such as grass and sky. As a result, visually discriminative objects may become sparse or absent. On the other hand, even when the model fails to maintain spatial consistency, it can still generate visually similar base scenes (i.e., combinations of grass and sky; see later frames in Fig. 5). Consequently, this leads to artificially higher consistency scores.

This exception does not affect our overall conclusions. Rather, it further underscores the importance of the "visually distinguishable" principle in dataset construction. At the same time, it points to a promising direction for future work, such as incorporating different weighting across trajectory segments to better account for varying scene complexity.

Table 2: **Evaluation Results of Four Baselines under Different Navigation Ranges**. The ✅ icon indicates that the model was trained on our dataset, while ❌ indicates using a pretrained checkpoint. ABA and ABCA denotes the navigation type, and the suffix of model(-5,-10...) denotes the navigation range.

Model	Train	SGCS ↑		SSIM ↑		LPIPS ↓		FVD ↓	
		ABA	ABCA	ABA	ABCA	ABA	ABCA	ABA	ABCA
Oasis-5	❌	**0.31**±0.24	**0.35**±0.28	0.36±0.13	0.34±0.12	0.76±0.09	0.82±0.11	2615±1067	2583±647
Oasis-15	❌	**0.26**±0.20	**0.27**±0.22	0.37±0.12	0.38±0.14	0.82±0.08	0.81±0.10	2516±567	3146±1055
Oasis-30	❌	**0.18**±0.15	**0.21**±0.21	0.33±0.11	0.35±0.11	0.86±0.08	0.85±0.09	3131±713	3199±1000
Oasis-50	❌	**0.11**±0.09	**0.20**±0.21	0.36±0.12	0.36±0.11	0.86±0.09	0.83±0.07	3334±658	3162±1245
Mineworld-5	❌	**0.41**±0.24	**0.45**±0.26	0.31±0.09	0.32±0.10	0.73±0.05	0.72±0.07	2089±1007	1914±660
Mineworld-15	❌	**0.37**±0.23	**0.42**±0.24	0.34±0.13	0.32±0.11	0.74±0.08	0.74±0.07	2367±770	2009±921
Mineworld-30	❌	**0.30**±0.24	**0.33**±0.23	0.33±0.13	0.28±0.09	0.77±0.08	0.77±0.08	2316±945	2094±1047
Mineworld-50	❌	**0.32**±0.23	**0.38**±0.26	0.31±0.16	0.32±0.12	0.78±0.12	0.75±0.10	2077±632	2144±898
DIAMOND-5	✅	**0.31**±0.26	**0.35**±0.26	0.40±0.10	0.37±0.09	0.75±0.09	0.79±0.09	3353±1242	3336±1392
DIAMOND-15	✅	**0.31**±0.23	**0.28**±0.22	0.38±0.10	0.39±0.10	0.78±0.08	0.79±0.09	3691±937	3302±1191
DIAMOND-30	✅	**0.20**±0.19	**0.24**±0.23	0.37±0.10	0.35±0.10	0.81±0.07	0.81±0.08	3708±1243	3473±1355
DIAMOND-50	✅	**0.18**±0.18	**0.22**±0.23	0.37±0.10	0.38±0.09	0.83±0.09	0.81±0.08	3249±833	2994±906
NWM-5	✅	**0.40**±0.26	**0.39**±0.30	0.33±0.11	0.31±0.09	0.64±0.05	0.67±0.05	1950±380	2240±664
NWM-15	✅	**0.38**±0.24	**0.40**±0.25	0.30±0.12	0.33±0.12	0.67±0.03	0.65±0.05	2132±916	2338±1010
NWM-30	✅	**0.38**±0.21	**0.36**±0.25	0.32±0.11	0.30±0.11	0.69±0.04	0.71±0.03	1893±1047	2437±429
NWM-50	✅	**0.35**±0.21	**0.39**±0.22	0.28±0.13	0.33±0.11	0.72±0.08	0.65±0.04	2715±883	1537±415

6 Limitations and Future Work

Exploration of More Advanced SGCS Variants. In this work, we adopt a relatively simple object-level matching strategy. However, several promising extensions may further improve the metric. First, different segments of a trajectory could be assigned different weights, allowing the metric to emphasize visually informative or spatially challenging regions. Second, beyond object-level

matching, future versions of SGCS could explicitly model and compare relational edges between objects, enabling more structured scene-level consistency evaluation. Finally, for objects belonging to the same category, consistency could also be evaluated at the shape or internal structure level, either through finer-grained segmentation or more detailed matching strategies.

Static structures in the dataset: The structures in our dataset (such as houses and terrain) are static, and we intentionally excluded moving objects like mobs. This decision was made to simplify the task of modeling spatial consistency. However, in real-world environments, the positions of vehicles, people, and other dynamic objects change over time. Maintaining spatial consistency in such dynamic environments is a more challenging task and a promising direction for future exploration. We leave this task for future work.

References

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Thirty-eighth Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.12399.

Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos, 2022. URL https://arxiv.org/abs/2206.11795.

Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models, 2024. URL https://arxiv.org/abs/2412.03572.

Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators, 2024. URL https://openai.com/research/video-generation-models-as-world-simulators.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments, 2024. URL https://arxiv.org/abs/2402.15391.

Shaofei Cai, Bowei Zhang, Zihao Wang, Xiaojian Ma, Anji Liu, and Yitao Liang. Groot: Learning to follow instructions by watching gameplay videos, 2023.

Shaofei Cai, Bowei Zhang, Zihao Wang, Haowei Lin, Xiaojian Ma, Anji Liu, and Yitao Liang. Groot-2: Weakly supervised multi-modal instruction following agents, 2024. URL https://arxiv.org/abs/2412.10410.

Shaofei Cai, Zhancun Mu, Anji Liu, and Yitao Liang. Rocket-2: Steering visuomotor policy via cross-view goal alignment, 2025a. URL https://arxiv.org/abs/2503.02505.

Shaofei Cai, Zihao Wang, Kewei Lian, Zhancun Mu, Xiaojian Ma, Anji Liu, and Yitao Liang. Rocket-1: Mastering open-world interaction with visual-temporal context prompting, 2025b. URL https://arxiv.org/abs/2410.17856.

Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2026. URL https://arxiv.org/abs/2511.16719.

Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset, 2018. URL https://arxiv.org/abs/1705.07750.

Boyuan Chen, Diego Marti Monso, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion, 2024. URL https://arxiv.org/abs/2407.01392.

PrismarineJS Contributors. mineflayer: Create minecraft bots with a stable, high-level javascript api. https://github.com/PrismarineJS/mineflayer, 2013–2024.

Decart, Julian Quevedo, Quinn McIntyre, Spruce Campbell, Xinlei Chen, and Robert Wachen. Oasis: A universe in a transformer, 2024. URL https://oasis-model.github.io/.

Jingtao Ding, Yunke Zhang, Yu Shang, Yuheng Zhang, Zefang Zong, Jie Feng, Yuan Yuan, Hongyuan Su, Nian Li, Nicholas Sukiennik, Fengli Xu, and Yong Li. Understanding world or predicting future? a comprehensive survey of world models, 2024. URL https://arxiv.org/abs/2411.14499.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL https://arxiv.org/abs/2010.11929.

Yilun Du, Mengjiao Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Brian Ichter, Pierre Sermanet, Tianhe Yu, Pieter Abbeel, Joshua B. Tenenbaum, Leslie Kaelbling, Andy Zeng, and Jonathan Tompson. Video language planning, 2023. URL https://arxiv.org/abs/2310.10625.

Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35: 18343–18362, 2022.

Shenyuan Gao, Jiazhi Yang, Li Chen, Kashyap Chitta, Yihang Qiu, Andreas Geiger, Jun Zhang, and Hongyang Li. Vista: A generalizable driving world model with high fidelity and versatile controllability, 2024. URL https://arxiv.org/abs/2405.17398.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. URL https://arxiv.org/abs/1410.5401.

Junliang Guo, Yang Ye, Tianyu He, Haoyu Wu, Yushu Jiang, Tim Pearce, and Jiang Bian. Mineworld: a real-time and open-source interactive world model on minecraft. *arXiv preprint arXiv:2504.08388*, 2025.

Saurabh Gupta, Varun Tolani, James Davidson, Sergey Levine, Rahul Sukthankar, and Jitendra Malik. Cognitive mapping and planning for visual navigation, 2019. URL https://arxiv.org/abs/1702.03920.

William H Guss, Brandon Houghton, Nicholay Topin, Phillip Wang, Cayden Codel, Manuela Veloso, and Ruslan Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. *arXiv preprint arXiv:1907.13440*, 2019.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/2de5d16682c3c35007e4e92982f1a2ba-Paper.pdf.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination, 2020. URL https://arxiv.org/abs/1912.01603.

Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models, 2022. URL https://arxiv.org/abs/2010.02193.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024. URL https://arxiv.org/abs/2301.04104.

Yining Hong, Beide Liu, Maxine Wu, Yuanhao Zhai, Kai-Wei Chang, Linjie Li, Kevin Lin, Chung-Ching Lin, Jianfeng Wang, Zhengyuan Yang, Yingnian Wu, and Lijuan Wang. Slowfast-vgen: Slow-fast learning for action-driven long video generation, 2024. URL https://arxiv.org/abs/2410.23277.

Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. Gaia-1: A generative world model for autonomous driving, 2023. URL https://arxiv.org/abs/2309.17080.

Nuri Kim, Obin Kwon, Hwiyeon Yoo, Yunho Choi, Jeongho Park, and Songhwai Oh. Topological semantic graph memory for image-goal navigation, 2022. URL https://arxiv.org/abs/2209.08274.

H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955. doi: https://doi.org/10.1002/nav.3800020109. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109.

Shalev Lifshitz, Keiran Paster, Harris Chan, Jimmy Ba, and Sheila McIlraith. Steve-1: A generative model for text-to-behavior in minecraft, 2024. URL https://arxiv.org/abs/2306.00937.

Willi Menapace, Stéphane Lathuilière, Sergey Tulyakov, Aliaksandr Siarohin, and Elisa Ricci. Playable video generation, 2021. URL https://arxiv.org/abs/2101.12195.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models, 2023. URL https://arxiv.org/abs/2209.00588.

Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard Lewis, and Satinder Singh. Action-conditional video prediction using deep networks in atari games, 2015. URL https://arxiv.org/abs/1507.08750.

Emilio Parisotto and Ruslan Salakhutdinov. Neural map: Structured memory for deep reinforcement learning, 2017. URL https://arxiv.org/abs/1702.08360.

William Peebles and Saining Xie. Scalable diffusion models with transformers. *arXiv preprint arXiv:2212.09748*, 2022.

Jan Robine, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. Transformer-based world models are happy with 100k interactions, 2023. URL https://arxiv.org/abs/2303.07109.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL https://arxiv.org/abs/1505.04597.

Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation, 2018. URL https://arxiv.org/abs/1803.00653.

Kiwhan Song, Boyuan Chen, Max Simchowitz, Yilun Du, Russ Tedrake, and Vincent Sitzmann. History-guided video diffusion, 2025. URL https://arxiv.org/abs/2502.06764.

Thomas Unterthiner, Sjoerd van Steenkiste, Karol Kurach, Raphael Marinier, Marcin Michalski, and Sylvain Gelly. Towards accurate generative models of video: A new metric & challenges, 2019. URL https://arxiv.org/abs/1812.01717.

Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines, 2024. URL https://arxiv.org/abs/2408.14837.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Qianqian Wang*, Yifei Zhang*, Aleksander Holynski, Alexei A. Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. In *CVPR*, 2025.

Xiaofeng Wang, Zheng Zhu, Guan Huang, Xinze Chen, Jiagang Zhu, and Jiwen Lu. Drivedreamer: Towards real-world-driven world models for autonomous driving, 2023a. URL https://arxiv.org/abs/2309.09777.

Zihan Wang, Xiangyang Li, Jiahao Yang, Yeqi Liu, and Shuqiang Jiang. Gridmm: Grid memory map for vision-and-language navigation, 2023b. URL https://arxiv.org/abs/2307.12907.

Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. Daydreamer: World models for physical robot learning, 2022. URL https://arxiv.org/abs/2206.14176.

Mengjiao Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023.

Sherry Yang, Jacob Walker, Jack Parker-Holder, Yilun Du, Jake Bruce, Andre Barreto, Pieter Abbeel, and Dale Schuurmans. Video as the new language for real-world decision making, 2024. URL https://arxiv.org/abs/2402.17139.

Yuncong Yang, Han Yang, Jiachen Zhou, Peihao Chen, Hongxin Zhang, Yilun Du, and Chuang Gan. 3d-mem: 3d scene memory for embodied exploration and reasoning, 2025. URL https://arxiv.org/abs/2411.17735.

Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

Guangyu Zhao, Kewei Lian, Haowei Lin, Haobo Fu, Qiang Fu, Shaofei Cai, Zihao Wang, and Yitao Liang. Optimizing latent goal by learning from trajectory preference, 2024. URL https://arxiv.org/abs/2412.02125.

Zheng Zhu, Xiaofeng Wang, Wangbo Zhao, Chen Min, Nianchen Deng, Min Dou, Yuqi Wang, Botian Shi, Kai Wang, Chi Zhang, Yang You, Zhaoxiang Zhang, Dawei Zhao, Liang Xiao, Jian Zhao, Jiwen Lu, and Guan Huang. Is sora a world simulator? a comprehensive survey on general world models and beyond. *arXiv preprint arXiv:2405.03520*, 2024.

Appendix Overview

Our appendix is organized as follows:

- **Appendix A** describes the platform setup for our data collection, including the Minecraft environment setup and three javascript plugins we modified to manipulate the bot and collect data.
- **Appendix B** presents detials of our dataset, including B.2 data storage format, B.3 statistical information and B.4 visualization of our LOOPNAV dataset.
- **Appendix C** Evaluation Pipeline Details. C.1 introduces our SGCS metric.
- **Appendix D** presents the detailed experimental settings and additional experimental results: **D.1**: Synthetic Perturbation Details. **D.2**: Open-oasis, **D.3**: Mineworld, **D.4**: DIAMOND, **D.5**: Navigation World Model.

A Platform Setup

We set up a local Minecraft server and use the Mineflayer platform to control the agent and collect trajectory data. For path planning, we employ the Mineflayer Pathfinder plugin, which computes shortest paths between waypoints using the A* algorithm. We use the Prismarine Viewer to render and visualize the agent's behavior and collected observations. To better support learning from the collected data and simulate human-like control dynamics, we make several key modifications to the default behavior of these plugins. These choices define some of the core characteristics of our dataset:

Restricted action space: The agent is limited to using only three actions during navigation — forward, jump, and camera rotation — closely matching the action primitives of a human player in Minecraft. See Figure 6 for demostrations.

Sequential action execution: At any given time step, only a single action is allowed. That is, the agent cannot rotate the camera and move forward simultaneously. This avoids entangled motion patterns and improves the clarity of spatial transitions in the data.

Smooth camera control: We impose a maximum angular velocity on camera rotations to prevent abrupt viewpoint changes between frames, ensuring more stable visual continuity for training.

Removal of irrelevant elements: To focus the dataset on pure navigation, we disabled all entity spawning (e.g., mobs), and removed UI elements such as the hotbar and hands, eliminating visual distractions and ensuring task purity.

A.1 Minecraft

Minecraft is an open-world environment characterized by diverse terrains and rich interaction dynamics. We selected Minecraft for the following reasons:

- **Rich environmental diversity**: Minecraft worlds are procedurally generated using random seeds, featuring a wide range of biomes and uniquely structured villages, enabling diverse and non-repetitive environments.
- **Extensive community resources**: Minecraft has a mature modding ecosystem that allows us to incorporate a wide variety of custom elements and tasks. In addition, there is a large amount of publicly available Minecraft gameplay footage on platforms like YouTube, which could be leveraged for future large-scale pretraining.
- **Rising research interest**: An increasing number of recent works have begun to study world modeling and agent learning in Minecraft, making it a timely and relevant platform for evaluating spatial consistency and memory mechanisms.
- **Efficient simulation**: Compared to real-world data collection, Minecraft enables faster, cheaper, and highly parallelizable simulation, making it well-suited for large-scale controlled data generation.

We established a local Mineworld server running **Java Edition** version **1.16.5**, utilizing the seed value of **42**. The server was configured in survival mode with the difficulty set to peaceful, and mob spawning was disabled to eliminate the presence of animals. This configuration was implemented to prevent interference from mobs during trajectory-related experiments.

We utilized the Chunkbase website [3] to locate various villages and biomes. A total of 120 villages were collected—comprising 6 types, with 20 samples each—along with locations of 18 distinct biomes and 8 structures. The agent was teleported to each initial position using Minecraft's teleport command before commencing navigation.

A.2 Mineflayer

Minefayer [4] is a powerful open-source JavaScript API designed for flexibly controlling in-game bots within Minecraft. It interacts with the Minecraft server by continuously reading block information around the bot based on its current x, y, and z coordinates, which is loaded locally. Users are allowed to specify actions for the bot to perform. Mineflayer simulates the bot's movement physically, computes its resulting state and position after each action, and subsequently sends updated coordinates (x, y, z), yaw, and pitch values to the server. Benefiting from this flexible interaction mechanism, diverse action policies can be developed to enable the agent to accomplish various tasks.

We introduced a modification to Mineflayer by constraining the horizontal rotation speed per time step, such that each action can result in a **maximum rotation of 0.1 radians**. This adjustment ensures smoother and more gradual viewpoint transitions.

A.3 Mineflayer-pathfinder

The Mineflayer Pathfinder [5] is an application built on top of Mineflayer. It is capable of planning a path from the bot's current position to a target coordinate (x, y, z) using the A* algorithm. The bot is then controlled through a series of actions to navigate toward the destination.

[3] http://www.chunkbase.com/

[4] https://github.com/PrismarineJS/mineflayer

[5] https://github.com/PrismarineJS/mineflayer-pathfinder

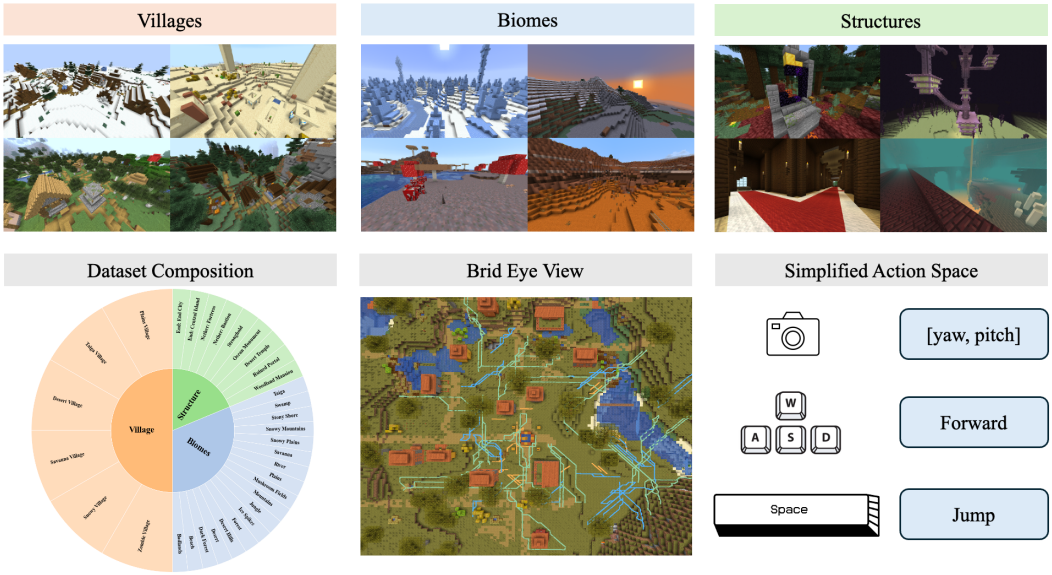


Figure 6: **Overview of the Minecraft Elements**. Top row, left to right: Examples of villages, biomes, and structures in Minecraft. Bottom row, left to right: composition of sampling locations in our dataset; a bird-eye view of real ABA-type exploration trajectories (display steps 5, 15, and 30 for simplicity); Simplified Action Space used for data collection and agent interaction.

In the original implementation, camera rotation and movement were coupled within the same action. In our modified version, at any given time step, **only a single action is allowed**. That is, the agent cannot rotate the camera and move forward simultaneously. This avoids entangled motion patterns and improves the clarity of spatial transitions in the data.

Furthermore, the original Mineflayer Pathfinder often exhibited sharp turns and other non-smooth behaviors during navigation, particularly causing rapid camera jitter near jagged block edges. To mitigate this issue, we prevent the bot from moving along the edges of 1×1 blocks, thereby eliminating such undesirable motion artifacts.

A.4 Prismarine-viewer

Prismarine Viewer [6] is a complementary plugin designed to work alongside Mineflayer, capable of rendering the Minecraft environment from either a first-person or third-person perspective. In this sense, the combination of Mineflayer and Prismarine Viewer effectively constitutes a functional Minecraft client. We employed Prismarine Viewer to render and generate gameplay visuals.

The rendering logic of the viewer operates by periodically capturing the current coordinates (x, y, z) yaw pitch and rendering the scene at a fixed time interval. We set this sampling frequency to 50 Hz, matching both Minecraft's default refresh rate and the action-sending frequency. As a result, each frame is rendered every 0.02 seconds, synchronized with the execution of a new action.

B Dataset details and Visualization

B.1 Overview

A total of 120 distinct villages were collected, comprising 6 types with 20 instances each. Our experiments were primarily conducted on this dataset of 120 villages. In addition, 18 different biomes and 8 types of locations were also gathered. Detailed information regarding the specific types of villages, biomes, and locations is provided in the table 3 below.

[6] https://github.com/PrismarineJS/prismarine-viewer

Table 3: Minecraft locations: Villages, Biomes, and Structures

Village	Biome		Structure
Plains Village	Badlands	Plains	Woodland Mansion
Savanna Village	Beach	River	Ruined Portal
Snowy Village	Dark Forest	Savanna	Desert Temple
Taiga Village	Desert	Snowy Plains	Stronghold
Desert Village	Desert Hills	Snowy Mountains	Nether Bastion
Forest Village	Forest	Stony Shore	Nether Fortress
Zombie Village	Ice Spikes	Swamp	End Mainland
	Jungle	Taiga	End City
	Mushroom Fields	Mountains	

For each location, we collected two types of trajectories: **{ABA, ABCA}**. For each type, trajectories were generated with four different navigation ranges: **5, 15, 30, and 50**. Each length comprises **20** distinct trajectories with varied start and end points.

This resulted in a total of 19,200 trajectories collected across all villages. With an average length of approximately 1,000 frames per trajectory, the dataset contains around **19.2 million** frames, equivalent to roughly **250 hours** of gameplay.

For experimental purposes, the villages were partitioned into training, validation, and test sets as follows: villages 1-16 were used for training, villages 17–18 for validation, and villages 19–20 for testing. The trajectories were not shuffled randomly across splits to prevent the model from memorizing structural features seen during training. Instead, this partitioning strategy encourages the model to reconstruct environmental structures from contextual information, as each village has a unique layout.

B.2 Data Format

The smallest unit in our dataset consists of a pair of files: an **.avi video file** and a corresponding **.json file**, both of equal length. The .avi file stores the visual observations, while the .json file contains the associated state and action data. To avoid introducing inter-frame dependencies, we use MJPG (Motion JPEG) compression instead of H.264. Each frame is stored independently, and the resolution of the images is 640×360 at BGR format. The value of each BRG pixel is from [0,255]. The videos are recorded at 20 frames per second (FPS). This frequency aligns with the physical tick rate of Minecraft, ensuring consistent synchronization between control and simulation.

One step of recoreded trajectories in .json have following keys:

- x: current x coordinate, rounded to three decimal places.
- y: current y coordinate, namely hight, rounded to three decimal places.
- z: current z coordinate, rounded to three decimal places.
- yaw: the agent's horizontal viewing angle, measured in radians, ranging from $-\pi$ to π, where 0 indicates the agent is facing the positive z-axis.
- pitch: the agent's vertical viewing angle, measured in radians, ranging from $-\pi/2$ to $\pi/2$, where 0 indicates the agent is looking straight ahead (parallel to the ground plane).
- action: a dictionary with three possible keys: forward, jump, and camera. The values of forward and jump are booleans indicating whether the corresponding action is executed. The camera key holds a tuple [yaw, pitch], representing the **change** in the agent's viewing angle. Our sampling strategy ensures that these three actions do not occur simultaneously.
- goal: a dictionary containing the target's coordinates, with keys x and z representing the target position on the x- and z-axes, respectively.
- frame count: the index of the current frame. Due to rendering initialization, the first 20 frames are skipped, so frame count starts at 20. Since we ensure that the entire trajectory is recorded properly, the actual frame index within the trajectory can be obtained by simply subtracting 20.

- extra info: seed, location, navigation type $\in \{ABA, ABCA\}$, navigation range $\in \{5, 15, 30, 50\}$.

B.3 Statistical Information

B.3.1 Trajectory Length

To better illustrate the scale and difficulty gradient of the dataset, the distribution of trajectory lengths is presented in the Figure 7 and Table 4 below.

From the perspective of sequence length, even for the shortest navigation range (i.e., range = 5), the average trajectory length reaches as high as 180 frames. This implies that during evaluation, models must be capable of attending to visual inputs from over 100 frames ago. However, most current world models are limited to a history (context window) of only 32 frames, indicating that the required temporal context in our setting substantially exceeds the capacity of existing models.

In terms of distribution, different navigation ranges exhibit minimal overlap in their length distributions, which supports the validity of our difficulty curriculum design.

Table 4: Mean Trajectory Frames ($\pm$ Std) for Different Trajectory Types

Trajectory Type	Range 5	Range 15	Range 30	Range 50
ABA	180.5 ± 30.4	356.5 ± 71.6	627.1 ± 130.3	967.8 ± 195.4
ABCA	251.0 ± 47.1	544.7 ± 113.1	968.1 ± 203.7	1362.6 ± 260.1

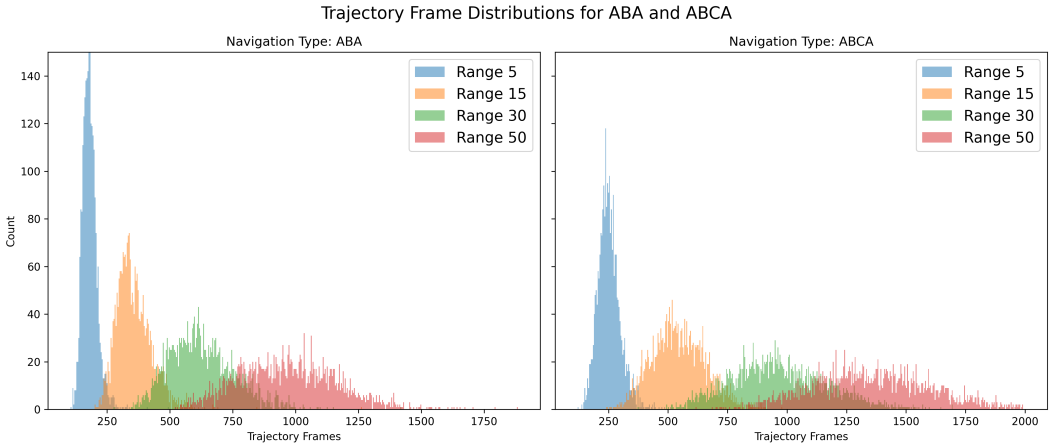


Figure 7: **Trajectory Frame Distribution of Navgation Type ABA and ABCA**

B.3.2 Trajectory Variance

In the original data collection pipeline, both the $A \to B$ and $B \to A$ paths were independently generated using A* search. Although this produced valid loop trajectories, the two paths could differ due to path-planning tie-breaking and local navigation artifacts. This introduces an additional source of trajectory variance that is unrelated to the intended spatial loop structure.

To reduce this confound, we revise the ABA collection protocol. The agent still uses A* search to navigate from A to B, preserving the same start-point sampling, target-point sampling, navigation ranges, and environment settings as before. However, for the return trajectory, instead of running A* again from B to A, the agent follows the recorded $A \to B$ waypoint sequence in reverse order. This reverse-waypoint return design encourages the $B \to A$ path to geometrically match the forward path as closely as possible, while still retaining physical execution, camera control, and rendering in the Minecraft environment.

We quantify the remaining discrepancy between the forward and return paths using the area enclosed by the loop. Let $\mathcal{A}$ denote the enclosed area and L denote the total trajectory length of the forward

and return paths. We report the normalized deviation as $2\mathcal{A}/L$, which corresponds to the average lateral deviation under a rectangular-loop approximation. The results are shown in Table 5.

Table 5: Deviation Between $A \to B$ and $B \to A$ Paths After Reverse-Waypoint Return

Metric	Range 5	Range 15	Range 30	Range 50
Enclosed Area	0.28 ± 0.36	0.54 ± 1.08	0.94 ± 3.26	1.66 ± 10.72
Normalized Deviation	0.046 ± 0.041	0.029 ± 0.062	0.025 ± 0.074	0.030 ± 0.196

Compared with independently planning both directions using A*, the reverse-waypoint return substantially reduces the geometric discrepancy between the forward and return paths. The enclosed area remains close to zero across all navigation ranges, even for the longest range of 50 grids. This indicates that the revised ABA trajectories better isolate the intended loop-closure behavior: the return phase revisits the previously traversed route rather than introducing a new path generated by a second A* search.

B.4 Dataset Visualization

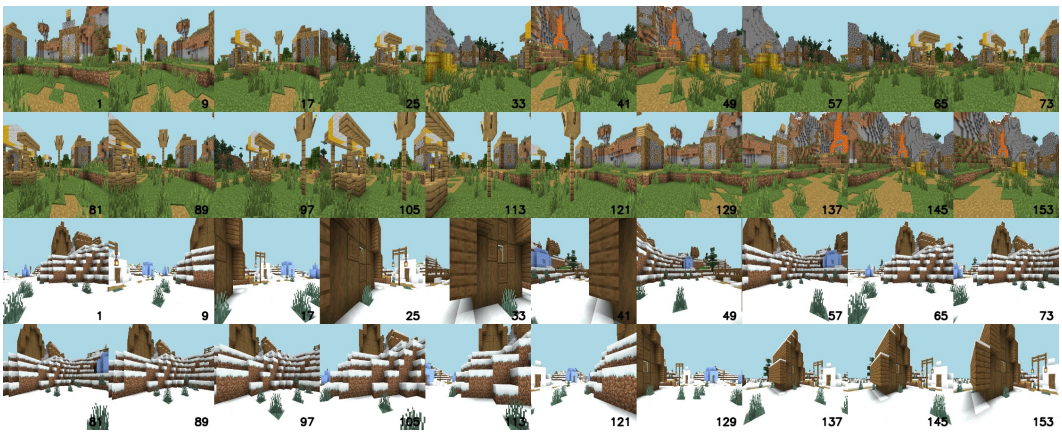


Figure 8: Demonstration of two LOOPNAV Trajectories. Upper half: Plains village. Lower Half: Snowy village. Black number indicates coresponding frame number.

We illustrate in Figure 8 two trajectories extracted from datasets with a navigation type of ABA and a navigation range of **5**. The upper subfigure corresponds to plains village (village ID:20, trajectory ID:05-10_14-09-44), while the lower subfigure is taken from snowy village (village ID:20, trajectory ID:05-10_17-46-35).

In the plains village, we observe that **frame 1 is visually similar to frame 129, and frame 41 is similar to frame 153.** In the snowy village, **frame 17 resembles frame 129, and frame 25 is similar to frame 153.** These observations suggest that, in order for a world model to accurately reconstruct future scenes, it must retain information from more than 100 frames earlier, and be capable of leveraging spatial context to infer and restore visual content.

To provide an intuitive visualization of the trajectories in our dataset, we present bird's-eye views of real trajectories. In Figure 9, the location is taken from plains village (village ID: 2). The left panel shows a trajectory of type ABA, while the right panel displays a trajectory of type ABCA. Yellow, blue, and green indicate navigation ranges of 5, 15, and 30, respectively. For visual clarity, trajectories with a navigation range of 50 are omitted.

C Evaluation Pipeline Details

C.1 SGCS Computation Details

We describe the full evaluation pipeline used to compute the Scene Graph Consistency Score (SGCS) between two videos. The implementation follows a frame-wise comparison protocol with object-level matching and category-wise aggregation.

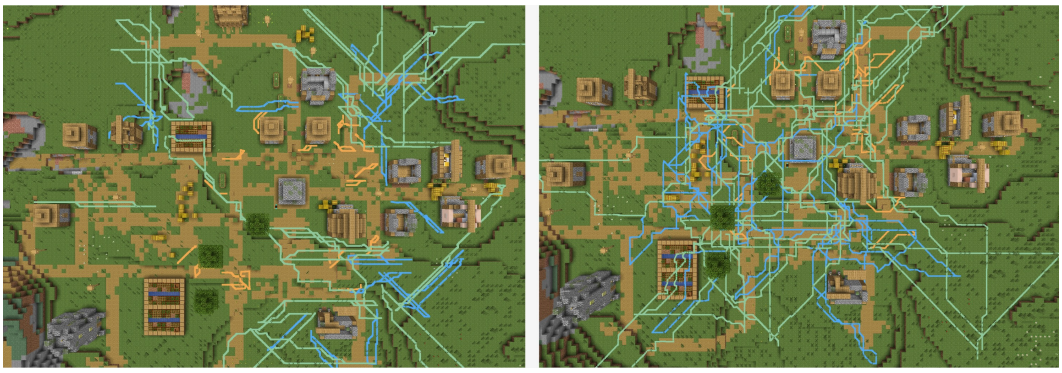


Figure 9: Demonstration of bird eye views for real dataset. Left: ABA type trajectories. Right: ABCA type trajectories. Yellow, blue, green indicate range 5, 15, 30 respectively.

Frame Sampling and Alignment. Given two videos A and B, we first sample frames with a fixed stride s (default $s = 5$). For each sampled frame index t, we extract the corresponding frame pair (I_t^A, I_t^B). If the two frames have different spatial resolutions, they are resized to a common resolution defined by the minimum height and width across the pair.

Object Segmentation via Text Prompts. For each frame, we extract object masks using a text-conditioned segmentation model (SAM3). We employ SAM 3.1 (released March 27, 2026) [7][8]for scene graph consistency evaluation. Specifically, we define a fixed set of semantic categories (we use *building, tree, flower, grass, path, water, sky, sand, dirt* for Minecraft), and for each category, we query the model to obtain a set of binary masks. Masks with low confidence (threshold 0.5 by default) are discarded. This produces, for each category c, a set of masks $\mathcal{M}_c^A$ and $\mathcal{M}_c^B$ for the two frames.

Mask Representation. Each mask is converted into a compact geometric representation consisting of:

- **Centroid:** computed as the mean of foreground pixel coordinates;
- **Area:** computed as the number of foreground pixels.

Masks with zero area are ignored. This representation allows efficient spatial comparison while remaining robust to shape irregularities.

Instance Matching. For each category c, we match instances across the two frames using centroid distance. Let $\{x_i^A\}$ and $\{x_j^B\}$ denote the centroids. We compute pairwise Euclidean distances and normalize them by the image diagonal $D = \sqrt{H^2 + W^2}$. A pair is considered a valid match if:

$$\|x_i^A - x_j^B\|_2 < \tau \cdot D,$$

where $\tau = 0.1$ by default. We then perform bipartite matching using the Hungarian algorithm to maximize the number of valid matches.

Category-Level Score. For each category c, let m and n denote the number of detected instances in the two frames, and p the number of matched pairs. We define the category-level consistency score as:

$$S_c = \frac{2p}{m + n}$$

This formulation corresponds to an instance-level F1 (Dice) score, penalizing both missing and mismatched objects.

[7]https://github.com/facebookresearch/sam3.git
[8]https://huggingface.co/facebook/sam3.1

Area-Weighted Aggregation. To account for object importance, we weight each category by its spatial extent:

$$w_c = \frac{\text{area}_c^A + \text{area}_c^B}{2}.$$

The frame-level SGCS is computed as:

$$\text{SGCS}_t = \frac{\sum_c w_c S_c}{\sum_c w_c}.$$

Video-Level Aggregation. Finally, we compute the video-level SGCS by averaging over all sampled frames:

$$\text{SGCS}_{video} = \frac{1}{T} \sum_{t=1}^{T} \text{SGCS}_t.$$

In dataset-level evaluation, we further report the mean and standard deviation across trajectories, where each trajectory-level score is obtained via temporal averaging.

C.2 Traditonal Evaluation Metrics

Fréchet Video Distance (FVD) We adopt Fréchet Video Distance (FVD) [Unterthiner et al., 2019] as a primary metric. FVD computes the distance between the distribution of real and generated video features extracted by an Inflated 3D ConvNet (I3D) [Carreira and Zisserman, 2018], capturing both spatial and temporal statistics at a high level.

Learned Perceptual Image Patch Similarity (LPIPS) We use LPIPS [Zhang et al., 2018] to evaluate the perceptual similarity between generated and ground truth frames. LPIPS leverages deep features from pretrained networks (e.g., VGG) to assess semantic-level differences, which aligns better with human judgment than pixel-wise metrics. It is robust to minor texture or color variations while remaining sensitive to object structure and layout.

Structural Similarity Index Measure (SSIM) SSIM is employed to complement LPIPS by quantifying structural fidelity and luminance similarity between generated and reference frames. While it is more sensitive to low-level details than LPIPS, it still emphasizes perceptual structure over exact pixel match.

C.3 Context Coverage in ABCA Trajectories

For longer $A \to B \to C \to A$ trajectories, we evaluate only the $C \to A$ segment, treating $A \to B \to C$ as the exploration context. It is worth noting that in these trajectories, the return path from $C \to A$ is not guaranteed to be fully covered by the earlier observations from exploration context. However, in practice, we constrain the exploration range such that the majority of $C \to A$ trajectories traverse areas that have been previously observed, albeit from different viewpoints or at different times(Figure 9 for bird-eye view illustrations). As a result, the model typically has access to sufficient contextual information to support the reconstruction of the return path, making our evaluation of spatial consistency meaningful.

D Experiment Settings and Results

We provide a detailed description of the experimental settings to facilitate reproduction. All inference and training experiments were conducted on NVIDIA GeForce RTX 4090 GPUs, with a total compute time of approximately 1000 GPU hours respectively.

D.1 Synthetic Perturbation Details

This experiment is conducted on the navigation dataset with navigation type ABA and sequence length 5, covering all 6 distinct villages. Since our evaluation applies synthetic perturbations directly to ground-truth frames, the navigation type and trajectory length do not significantly affect the results. For reporting variability, we compute the standard deviation over video-level scores, where each video-level score is obtained by averaging frame-level metrics within a single video. We introduce all the synthetic perturbation details below, and the demonstration is in Figure 10.



Figure 10: Synthetic Perturbation Demonstrations.

Color change. For the color-change perturbation, we modify only the appearance of the input frame while preserving its spatial layout. Specifically, each RGB frame is first adjusted by applying a contrast multiplier of 1.12 and adding a brightness offset of 18.0 to all pixel values. The result is clipped to the valid range $[0, 255]$. We then convert the image to HSV space and further increase the saturation channel by a factor of 1.15, followed by conversion back to RGB. This perturbation changes low-level color and illumination statistics without modifying object geometry or object layout.

Small translation. For the small-translation perturbation, we apply a global 2D shift to the entire frame. The horizontal and vertical displacements are set to 4% of the frame width and height, respectively. Formally, for a frame of size $W \times H$, the translation offsets are $\Delta x = 0.04W$ and $\Delta y = 0.04H$, rounded to integer pixel values with a minimum displacement of one pixel. The translated frame is generated using bilinear interpolation, and missing boundary regions are filled using reflection padding. This perturbation introduces pixel-level misalignment while preserving the relative spatial arrangement of objects.

Small rotation. For the small-rotation perturbation, we rotate the entire frame around its image center by 5° while keeping the image resolution unchanged. The transformation is implemented using an affine rotation matrix with scale fixed to 1.0. Bilinear interpolation is used for resampling, and reflection padding is used to fill boundary regions. Since all objects undergo the same rigid transformation, the global image alignment changes, but the underlying scene structure and object relationships are largely preserved.

Scaling via crop-and-resize. For the scaling perturbation, we simulate a zoom-in transformation using center crop followed by resizing. Specifically, we take a centered crop covering 90% of the original frame height and width, and then resize the cropped region back to the original resolution. This produces a mild global scaling effect while maintaining the main scene layout. Because the transformation affects the whole frame consistently, it should not substantially alter object-level spatial consistency, although pixel-level similarity metrics may be affected by the resampling and scale change.

Object deletion. For the object-deletion perturbation, we first obtain object masks using SAM with the predefined text categories. Candidate object masks are filtered by area: masks smaller than 0.2% of the image area or larger than 40% of the image area are discarded. Duplicate masks are removed using an IoU threshold of 0.85. Among the remaining candidates, the largest object mask is selected for deletion. The selected mask is dilated with a 5×5 kernel and removed using Telea inpainting. This perturbation directly violates spatial consistency by removing a detected object from the scene while keeping the surrounding visual content plausible.

Object position swapping. For the object-swapping perturbation, we again use SAM-derived object masks filtered with the same area constraints: minimum area fraction 0.002, maximum area fraction 0.4, and duplicate removal with IoU threshold 0.85. If at least two valid objects are found, we select the two largest object candidates. The union of their masks is first removed from the image using inpainting. Then, each object patch is pasted into the other object's bounding box. During pasting, the source patch and source mask are resized to match the target bounding box, and only pixels inside the resized mask are copied. This perturbation preserves the presence of objects but changes their spatial locations, thereby creating a structural inconsistency in object layout.

D.2 Open Oasis

D.2.1 Oasis Settings

We use the open-sourced Oasis-500M model for inference only. We do not train the Oasis model ourselves, as it is already pretrained on Minecraft VPT contractor data, and the official training code has not been released. We use 32 frames as the conditioning input, wthich is the maximum supported context length, and perform inference in an auto-regressive manner.

Their observation space is defined as (640, 360, 3), which is fully consistent with our dataset and thus requires no additional modifications. Their action space follows VPT's CameraQuantizer, with a maximum value of 20 and a bin size of 0.5, resulting in 40 discrete buckets that are normalized to the range [-1, 1]. In contrast, our actions are defined in radians within the range [-0.1, 0.1], and we directly use the raw values as actions. Additionally, their definitions of cameraX and cameraY are reversed compared to ours(and VPT's). Other actions, such as forward and jump, are consistent between the two settings. Inference hyperparameters are shown in Table 6

Table 6: Oasis Inference Hyperparameters

Model	**Video Encoder**	**Sampling**
Model: `oasis-500M`	Model: ViT-VAE-L/20	DDIM steps: 10
Structure: DiT-S/2	VAE patch size: 20	Max noise level: 1000
Model.max_frames: 32	# Prompt frames: 32	Noise absolute max: 20
		Stabilization level: 15
		Beta schedule: Sigmoid

D.2.2 Oasis Results

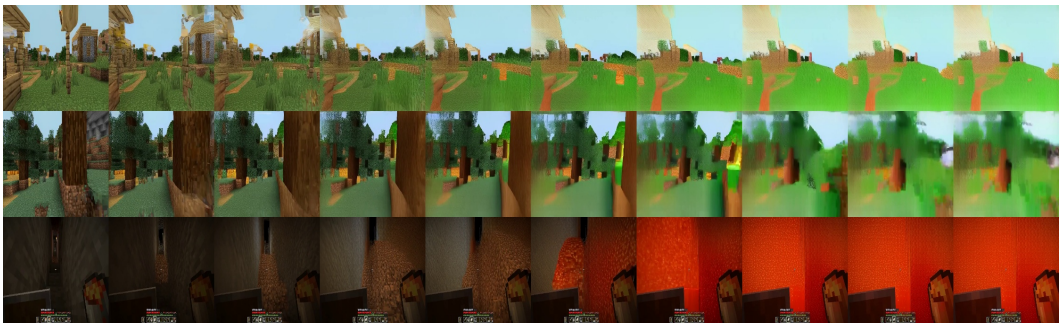


Figure 11: Results of Oasis. The first row corresponds to plains village, the second row to taiga village, and the third row shows the official inference demo. All three cases exhibit increasing blurriness.

However, the results fall short of expectations. As shown in Figure 11, we observe that errors compound over time: small imperfections quickly snowball into collapsed frames. After approximately 30 frames, the generated images become increasingly blurred and fail to recover. We include a sample from plains village and a sample from taiga village. To demonstrate that the blurriness is not caused by our dataset or action configuration, we additionally include the official inference demo (Player729-f153ac423f61-20210806-224813, 256 frames) in line 3 for comparison.

D.3 Mineworld

D.3.1 Mineworld Settings

Mineworld is also a model pretrained on Minecraft VPT contractor data, and its official training code has not been released. For the same reason, we directly evaluate Mineworld on our benchmark without further training. Specifically, we evaluate the largest publicly available model, Mineworld-1200M-16f and Mineworld-1200M-32f, which support 16 frames and 32 frames as context respectively.

Mineworld supports up to 32 frames of total context, which includes both historical and previously generated frames. Its generation process is chunk-based. We experiment using sliding window strategies: using 31 frames as context to autoregressively generate the remaining frames.

Mineworld defines its observation space as 384×224. To align with this, we resize our original 640×360 frames to 384×224 using an area-based interpolation method, and apply standard normalization to the pixel values. In Mineworld, visual observations are further compressed by a VAE into a latent representation of size 24×14. As for the action space, since Mineworld uses degrees to represent camera rotations, we convert our radian-based actions into degrees before passing them to the model.

D.3.2 Mineworld Results

To qualitatively evaluate the model's predictions, we visualize rollout results alongside ground truth frames in Figure 12. The first and third rows show rollouts for plains village (village ID:20, trajectory ID: 05-10_14-09-18) and desert village(village ID:20, trajectory ID: 05-10_14-11-32), respectively, while the second and fourth rows present their corresponding ground truth trajectories. We observe that, unlike Oasis, Mineworld does not exhibit visual collapse. However, it similarly lacks spatial consistency and fails to reconstruct the corresponding scene structure.

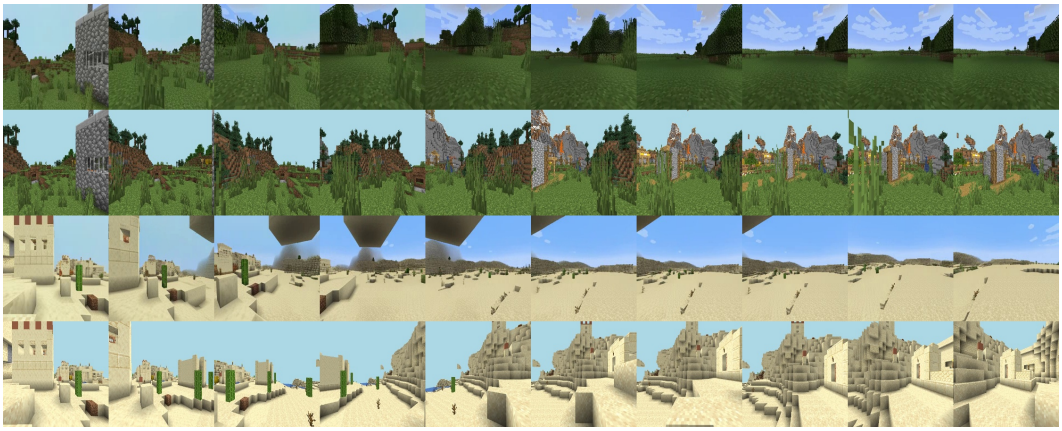


Figure 12: Results of Mineworld.The first and second rows show the rollout and ground truth for plains village, respectively; the third and fourth rows show the same for desert village.

D.4 DIAMOND

D.4.1 DIAMOND Settings

DIAMOND [Alonso et al., 2024] is a diffusion-based world model built upon the UNet architecture [Ronneberger et al., 2015]. It generates video frames autoregressively, conditioning on both past observations and actions, allowing it to model complex temporal dynamics in sequential decision-making environments. In its original design, DIAMOND is used as part of a model-based reinforcement learning pipeline, where the learned world model is used to train an RL agent and evaluated on the Atari 100k benchmark. And in CS:GO branch, DIAMOND also exhibits extraordinary ability to modeling dynamics. We follow the `CS:GO` configuration and focus solely on training the world model, omitting the reinforcement learning (RL) agent training phase. For the observation space, following the default settings of the CS:GO branch in DIAMOND, we first resize the input images from 640×360 to 320×180, and then further downscale them to 64×36. Diffusion is performed

on the 64×36 images, and an additional upsampler is trained to reconstruct the images back to 320×180.

For the action space, we adopt VPT-style camera quantization with a maximum value of 0.1 and a bin size of 0.02, resulting in 11 discrete bins for both yaw and pitch. Combined with forward and jump actions, the total action dimension is 24. The training hyperparameters are summarized in Table 7.

Table 7: Model Configuration Parameters

Category	Parameter	Denoiser	Upsampler
General	sigma_data	0.5	0.5
	sigma_offset_noise	0.1	0.1
	noise_previous_obs	true	false
	upsampling_factor	null	5
Inner Model	img_channels	3	3
	num_steps_conditioning	4	1
	cond_channels	2048	2048
Training	num_autoregressive_steps	4	1
	start_after_epochs	0	0
	steps_first_epoch	400	400
	steps_per_epoch	400	400
	sample_weights	null	null
	batch_size	64	16
	grad_acc_steps	2	2
	lr_warmup_steps	100	100
	max_grad_norm	10.0	10.0
Optimizer	lr	1e-4	1e-4
	weight_decay	1e-2	1e-2
	eps	1e-8	1e-8
Diffusion Sampler	num_steps_denoising	3	10
	sigma_min	2e-3	1
	sigma_max	20.0	5.0
	rho	7	7
	order	1	1
	s_churn	0.0	10.0
	s_tmin	0.0	1
	s_tmax	∞	5
	s_noise	1.0	0.9
	s_cond	0.005	0

D.4.2 DIAMOND Results

In Figure 13, we present the rollout results of DIAMOND. We observe that DIAMOND does not suffer from visual collapse and is able to reconstruct the initial frames relatively accurately, which aligns with its context window of length 32. However, as the rollout progresses, the generated scenes gradually converge to empty grasslands, indicating that the model forgets previously observed structures. This suggests that DIAMOND, like others, fails to maintain spatial consistency over long horizons.



Figure 13: Results of Mineworld. First row: Rollout results. Second row: Ground truth.

D.5 Navigation World Model

D.5.1 NWM Settings

Navigation World Model (NWM) proposes a world model based on the Conditioned Diffusion Transformer (CDiT). Regarding the observation space, we first align with NWM's input format by decomposing trajectory videos into individual images. Following the official setting, each image is resized to 224×224, normalized, and then compressed into a 32×32 latent representation using the Stable Diffusion VAE.

For the action space, NWM differs from prior models in that it conditions on the agent's (x, z) position and yaw angle, using absolute location and orientation to reconstruct future observations. Since our dataset records the (x, z) coordinates and yaw at each step, we can directly adopt this format. Note that pitch information is omitted here. Empirically, we find that for navigation tasks, pitch tends to be less critical than yaw.

We train a CDiT-L/2 model with a context window of 32 frames. Detailed training hyperparameters are provided in Table 8.

Table 8: NWM Training Configuration

Parameter	**Value**
Batch size	8
Number of workers	12
Model	CDiT-L/2
Learning rate	8×10^{-5}
Normalize action space	True
Gradient clipping value	10.0
Context size	32
Distance Prediction	
Min distance category	-64
Max distance category	64
Action Output	
Predicted trajectory length	64
Dataset Settings	
Image size	224
Goals per observation	4

D.5.2 NWM Results



Figure 14: Results of NWM. First row: Rollout results. Second row: Ground truth.

In Figure 14, we present the rollout results of Navigation World Model. We observe that NWM retains a considerable amount of memory in the initial frames. However, the generated frames are noticeably blurry and distorted, suggesting that the model is unable to faithfully reconstruct spatial details or maintain long-range spatial consistency.

E Extra Related Works

E.1 World Models

The goal of a world model is to simulate the environment: Given the current state and action, it predicts the next state and the corresponding reward. World models were originally proposed to improve sample efficiency in reinforcement learning [Oh et al., 2015, Ha and Schmidhuber, 2018]. In the context of model-based reinforcement learning, numerous studies have explored various architectural designs of world models. Dreamer [Hafner et al., 2020, 2022, 2024] uses a Recurrent State Space Model (RSSM) and achieves human-level performance on Atari games. TWM [Robine et al., 2023] adapts DreamerV2's RSSM to use a transformer architecture. IRIS [Micheli et al., 2023] builds image tokens with discrete autoencoder and adopts an autoregressive transformer. DIAMOND [Alonso et al., 2024] leverages diffusion models to generate future frames. These architectures, to varying degrees, retain information from the past to aid in future image prediction; however, they still lack an effective memory design capable of maintaining long-horizon spatial consistency.

Beyond model-based reinforcement learning, the potential of world models has also been increasingly explored in other domains. Playable Video Generation [Menapace et al., 2021], gameNGen [Valevski et al., 2024], Oasis [Decart et al., 2024] investigate the potential of using world models as game engines. GAIA [Hu et al., 2023], DriveDreamer Wang et al. [2023a], vista [Gao et al., 2024] explores their application in autonomous driving. In robotics, two of the most significant challenges are the scarcity of training data and the high cost of model training. The emergence of world models offers a promising direction for addressing these two major challenges. UniSim [Yang et al., 2023] learns a universal simulator for robot manipulation. DayDreamer [Wu et al., 2022] employs world model for real-world robot learning. NWM [Bar et al., 2024] investigates visual navigation problem in world model. In almost all downstream tasks—especially those involving decision-making such as autonomous driving and navigation—maintaining spatial consistency is a critical requirement.

From a more unified perspective, world models can be viewed as action-conditioned video generation models. In computer vision, generating videos has been a long standing challenge [Yang et al., 2024]. Recent approaches leverage transformers, and diffusion models [Chen et al., 2024, Song et al., 2025] to generate longer, more coherent video sequences. From the perspective of video modeling, maintaining spatial consistency across frames—especially over long temporal horizons—remains a critical and unsolved challenge.

E.2 Minecraft as an AI Testbed

Minecraft is an open-world environment characterized by diverse terrains and rich interaction dynamics. As one of the most popular games globally, it benefits from extensive community-driven resources and an active user base. Recent research has increasingly adopted Minecraft as a platform for training generative agents [Baker et al., 2022, Lifshitz et al., 2024, Cai et al., 2023, 2024, 2025b,a, Zhao et al., 2024] and constructing digital world models [Guo et al., 2025, Decart et al., 2024, Hong et al., 2024, Song et al., 2025]. Several Minecraft-based datasets and benchmarks have been introduced, including MineDoJo [Fan et al., 2022], which provides multimodal knowledge and human gameplay videos sourced from YouTube; VPT [Baker et al., 2022], which releases large-scale trajectories of human players engaged in building and mining; and MineRL [Guss et al., 2019], which focuses on trajectories related to cave exploration. However, these efforts primarily emphasize prospective exploration and object interaction, with limited attention to the role of historical context. In contrast, our proposed dataset and benchmark are designed to promote and assess models' abilities to leverage past observations and perform spatial reasoning. Our data collection pipeline is built on top of Mineflayer [Contributors, 2013–2024], a widely used rule-based Minecraft automation framework.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Our main claim is that spatial consistency is a critical yet under-addressed challenge in current world models. In this paper, we introduce a loop-based navigation dataset and benchmark specifically designed to evaluate and improve spatial consistency.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We include a part to discuss our limitation in Section 6.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We release our code and dataset in the link.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We release our code and dataset in the link.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: We include details of experiments in Section 5.3 and Appendix D.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: We report std of the expriment results.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)

- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We provide GPU useage in Appendix D.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: Yes, the research presented in the paper fully complies with the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: There is no societal impact of the work performed.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The datasets released in this paper do not present a high risk for misuse.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Yes, all external assets used in this work—including code, datasets, and pretrained models—are properly credited.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Yes, all external assets used in this work—including code, datasets, and pretrained models—are properly credited.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Core method development in this research does not involve LLMs as any important, original, or non-standard components.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.